KAVACH
ANTIVIRUS
Powered by Cecil Baraik

Technology Notes

What Kavach is built with, and how it works

Kavach Antivirus v1.1.2 · Windows 10 / 11

Developed by Cecil Baraik

github.com/cecilbaraik19/Kavach · November 2026

Contents

1. Kavach in one page
2. Programming language and platform
3. How Kavach is organised
4. Libraries and packages
5. Windows features and APIs used
6. How Kavach finds malware
7. How Kavach protects in real time
8. Security of Kavach itself
9. Data storage
10. Updates: signatures and the app
11. Building, installing and publishing
12. Tools used
13. Limits of Kavach
14. Key terms

1. Kavach in one page

Kavach is a real antivirus for Windows, written in **C#** on **.NET 8**. It has three parts: a **desktop app** people click on, a **Windows service** that protects the PC all the time, and a shared **engine library** that does the actual scanning. About 14,000 lines of code.

Area	Technology
Language	C# 12
Platform	.NET 8 (net8.0-windows), self-contained, ReadyToRun compiled, 64-bit
Desktop interface	WPF (Windows Presentation Foundation) with XAML; Windows Forms only for the tray icon
Background protection	Windows Service built with Microsoft.Extensions.Hosting.WindowsServices
Detection	SHA-256 hashes, own YARA-compatible rule engine, PE heuristics, script analysis, Authenticode checks
Archives	SharpCompress 0.50.8 (ZIP, RAR, 7z, TAR, GZ)
Windows APIs (P/Invoke)	WinVerifyTrust, IP Helper (GetExtendedTcpTable/UdpTable), process I/O counters, process security
App ↔ service	Named pipes
Encryption	AES (quarantine vault), PBKDF2-SHA256 (settings password), SHA-256 (fingerprints)
Installer	Inno Setup 6 + sc.exe (service registration)
Updates	ReversingLabs YARA rules (GitHub, MIT) and GitHub Releases API for app updates
Tools	Visual Studio, .NET SDK, Git, GitHub, PowerShell

2. Programming language and platform

C# 12

C# is Microsoft's main language for Windows software. It is fast, type-safe and has built-in memory management (garbage collection), so mistakes like buffer overflows are much harder to make — important for software that opens untrusted files all day. It can also call Windows' own C functions directly (P/Invoke).

.NET 8

- **Target**: `net8.0-windows`, because Kavach uses Windows-only features.
- **Self-contained**: the .NET runtime is shipped inside the installer, so the user does not need to install .NET.
- **ReadyToRun**: code is compiled to machine code ahead of time, so Kavach starts faster.
- **win-x64**: built for 64-bit Windows 10 and 11.

Why not Java or Python?

An antivirus must talk closely to Windows: services, file events, processes, digital signatures, network tables and the event log. C#/.NET does this natively. Java and Python need extra layers, start slower and make a proper Windows service and installer harder.

3. How Kavach is organised

The Visual Studio solution `Kavach.sln` has three projects:

Project	Output	Job
Kavach.Core	Kavach.Core.dll (class library)	The brain: scan engine, rules, quarantine, protection modules, settings, logs, updates. Used by both other projects.
Kavach.App	Kavach.exe (WPF app)	The window: 13 pages, tray icon, notifications, dialogs, light/dark theme.
Kavach.Service	KavachService.exe (Windows service)	Always-on protection: real-time, ransomware, USB, script memory, schedules, self-protection, daily rule updates.

How the parts work together

- The **service** starts with Windows (as SYSTEM) and runs the protection modules through **ProtectionHost**.
- When it finds a threat it sends an event over a **named pipe** (`Kavach.Events`); the **app** listens and shows a notification.
- Both share settings and data in `C:\ProgramData\Kavach`. The service notices changed settings with a **FileSystemWatcher** and applies them at once.
- If the service is not running, the app can run protection itself — but never both at the same time.
- The app is **single-instance**: opening it twice brings the first window forward.

Folders inside Kavach.Core

Folder	Contents
Engine	ScanEngine, HashDatabase, RuleEngine, PeAnalyzer, ScriptAnalyzer, ArchiveScanner, SignatureVerifier, FileClassifier, ScanCache
Protection	RealTimeMonitor, RansomwareGuard, ScriptMemoryMonitor, UsbMonitor, SelfProtection, Scheduler, ProtectionHost
Response	QuarantineManager, ProcessKiller
SystemChecks	StartupScanner, HostsFileChecker, ProcessScanner, NetworkMonitor
Scanning	ScanJob (one scan with progress, pause, cancel), ScanLocations
Infrastructure	Settings, ActivityLog, HistoryStore, ReportExporter, SignatureUpdater, AppUpdater, VirusTotalClient, EventPipe, AppPaths
Native	NativeMethods — every Windows API declaration

4. Libraries and packages

Library	Used for
WPF + XAML (.NET)	The whole desktop interface: pages, cards, styles, dark and light themes (ResourceDictionary swapping).
Windows Forms (.NET)	Only the system-tray icon (NotifyIcon), its menu and balloon notifications.
Microsoft.Extensions.Hosting.WindowsServices 8.0.1	Turns a .NET worker program into a real Windows service (start, stop, auto-restart).
SharpCompress 0.50.8	Opens ZIP, RAR, 7z, TAR and GZ archives to scan the files inside. Upgraded from 0.38.0 to remove a known vulnerability.
System.Diagnostics.EventLog 8.0.1	Reads the PowerShell event log live for script memory scanning.
System.Text.Json	Saves settings, history, quarantine index and caches as JSON files.
System.Security.Cryptography	SHA-256 hashing, AES encryption, PBKDF2 password hashing.
System.Net.Http (HttpClient)	Downloads rules and app updates; VirusTotal lookups.
System.IO.Pipes	Named pipes between service and app (with an ACL so normal users can read events).
Own PDF writer (SimplePdf)	Writes PDF scan reports without any extra library.

Few outside packages on purpose

Only three NuGet packages are used. Fewer dependencies mean fewer security risks — important for security software. The YARA engine, PE parser and PDF writer are Kavach's own code.

5. Windows features and APIs used

Windows feature / API	What Kavach uses it for
Windows Service Control Manager (sc.exe)	Installs KavachService as an automatic service that restarts if it fails.
FileSystemWatcher (ReadDirectoryChangesW)	Notices new and changed files for real-time protection and ransomware bait files.
WinVerifyTrust (wintrust.dll)	Checks Authenticode digital signatures: is a program really signed by Microsoft, Google …?
Catalog signatures	Recognises Windows' own files that are signed through catalog files.
GetExtendedTcpTable / GetExtendedUdpTable (iphlpapi.dll)	Lists every network connection with its owning process for the Network Monitor.
GetProcessIoCounters (kernel32.dll)	Finds which program is writing the most data — the likely ransomware.
Process security descriptor (SDDL)	Self-protection: normal users and programs cannot kill the Kavach service.
Event Log — PowerShell event 4104	Script Block Logging: PowerShell records every script it runs, already decoded.
Registry	Startup Run keys, services, scheduled tasks, the right-click menu, and turning on script block logging.
DriveInfo	Notices removable (USB) drives being plugged in.
Windows Defender cmdlets (Add-MpPreference)	The installer adds Kavach to Controlled Folder Access's allowed list.
DWM API (DwmSetWindowAttribute)	Dark or light window title bar to match the theme.
Mutex + named pipe	Single-instance app.

6. How Kavach finds malware

Every file goes through several **detection layers**. One strong hit is enough to flag a file.

Layer	How it works
1. Hash check (SHA-256)	Kavach computes the file's SHA-256 fingerprint and looks it up in its hash lists (EICAR, WannaCry, imported and MalwareBazaar hashes). An exact match = known malware.
2. YARA-compatible rules	Kavach's own rule engine reads .yar files: text, hex and regex strings with *nocase, wide, ascii, fullword*, conditions such as *any of, all of, at, filesize, uint16/uint32*. 15 own rules + 252 ReversingLabs rules.
3. PE heuristics	For .exe/.dll files Kavach reads the PE header and sections and scores warning signs: packed or encrypted code (entropy above 7.2), injection and keylogger APIs, odd section names, fake timestamps. Score 45 = Medium, 70 = High.
4. Script analysis	PowerShell, VBS, JS, BAT, HTA and LNK files are checked for downloads, encoded commands, obfuscation and disabling security.
5. File name tricks	Fake system file names, double extensions (invoice.pdf.exe) and RTLO characters that reverse the name.
6. Archive scanning	Files inside ZIP, RAR, 7z, TAR and GZ are scanned one by one, with limits against zip bombs (500 entries, 200 MB in total).
7. Digital signatures	Files properly signed by a trusted publisher skip the heuristic and rule checks — this greatly reduces false alarms.

Speed tricks

- **Scan cache**: unchanged clean files are skipped until the signatures change.
- **Smart skipping**: big media files cannot run code, so they are skipped.
- **Scan speed setting**: Low / Balanced / Fast controls how many CPU cores are used.
- **Developer folders** (node_modules, site-packages …): file-name tricks are not reported there, but contents are still scanned.

Threat levels

LOW → MEDIUM → HIGH → CRITICAL. High and Critical are quarantined automatically; Critical (such as ransomware) also stops the program at once.

7. How Kavach protects in real time

Module	Technique
Real-time protection	FileSystemWatcher on Downloads, Desktop, Documents, temp and startup folders. Each new or changed file is scanned about 1–5 seconds after it is saved.
Ransomware protection	Hidden bait files (!!!Kavach-Bait…) in Documents, Desktop and Pictures. If one changes, Kavach compares process I/O counters, finds the program writing the most data, kills it and quarantines it. It also watches for mass renames to new extensions.
Script memory scanning	Turns on PowerShell Script Block Logging, then reads event 4104 live with EventLogWatcher. Scripts arrive already decoded (even -EncodedCommand). Each one goes through the script analyzer and YARA rules; malicious ones get their PowerShell process stopped.
USB auto-scan	Watches the list of removable drives; a new drive is scanned at once, and autorun.inf tricks are blocked.
Download warnings	New programs in Downloads are checked for a digital signature, and the user is told.
Schedules	A scheduler in the service runs Quick/Full/Custom scans at set times, catching up if the PC was off.
Self-protection	The service's process gets a protected security descriptor, so normal users cannot terminate it.
Network monitor	IP Helper tables + an IP blacklist; connections to bad addresses are flagged.

8. Security of Kavach itself

- **Quarantine vault**: files are encrypted with **AES** (random IV per file, header "KAVACHQ1"), so a quarantined virus can never be run by accident.
- **Settings password**: never stored as text — only a **PBKDF2-SHA256** hash with a random salt and 100,000 iterations.
- **Update safety**: a downloaded Kavach installer is accepted only if its SHA-256 matches the fingerprint GitHub reports; otherwise it is deleted and never run. Only https://github.com download addresses are allowed.
- **Quoted service path**: the service is registered with a quoted path, avoiding the “unquoted service path” attack.
- **Least privilege**: the app runs as a normal user; only the service runs as SYSTEM. Admin rights are requested only when needed.
- **Data folder permissions**: C:\ProgramData\Kavach is set up by the installer.
- **VirusTotal privacy**: only the SHA-256 fingerprint is sent, never the file.

9. Data storage

Everything lives in `C:\ProgramData\Kavach` as simple JSON or text files — no database is needed.

File / folder	Contents
settings.json	All preferences, schedules, allowlist and the password hash
history.json	Scan history with every detection
activity.log	The activity log
Quarantine\ (+ index.json)	Encrypted quarantined files and their details
Signatures	Hash lists (.txt) and rule files (.yar), version.json
scancache.json	Fingerprints of files that were clean
Backups\, Updates	Backups of changed startup items / hosts file; downloaded app updates

10. Updates: signatures and the app

Detection rules

- Source: **ReversingLabs YARA rules** on GitHub (MIT licence, free, no account).
- Kavach downloads the repository as one ZIP, keeps Windows rules that use no YARA modules and that its engine fully supports (252 rules), and saves them as `online_reversinglabs.yar`.
- The service checks every 6 hours and downloads at most once a day.
- Optional: MalwareBazaar recent hashes (abuse.ch) with a free Auth-Key.

Kavach updating itself

1. Once a day the app asks the **GitHub Releases API** for the latest release of cecilbaraik19/Kavach.
2. It compares the release tag (e.g. v1.1.2) with its own version.
3. If newer, it downloads **KavachSetup-x.y.z.exe** and checks its SHA-256 against GitHub's digest.
4. It starts the installer silently; the installer stops the service, updates files, restarts the service and reopens Kavach.

Version number

One file, `Directory.Build.props`, holds the version for all three projects and the installer.

11. Building, installing and publishing

Build

- **BUILD-INSTALLER.bat** checks for the .NET SDK and Inno Setup (installs Inno Setup with winget if missing), then runs **build.ps1**.
- **build.ps1** runs `dotnet publish` for the app and the service (Release, win-x64, self-contained, ReadyToRun), then compiles the installer with Inno Setup's **ISCC**.
- Output: `installer\Output\KavachSetup-<version>.exe`.

Installer (Inno Setup 6)

- Installs to C:\Program Files\Kavach; publisher “Cecil Baraik”; needs admin rights.
- Registers **KavachService** with sc.exe (automatic start, restart on failure) and starts it.
- Adds Kavach to Windows Controlled Folder Access, creates shortcuts, and fully removes the service on uninstall.

Source control and release

- Code is on **GitHub**: github.com/cecilbaraik19/Kavach (public).
- **PUSH-TO-GITHUB.bat** commits all changes as a new commit and pushes them.
- Each version is a **GitHub Release** (v1.1.0, v1.1.1, v1.1.2) with the installer attached.

12. Tools used

Tool	Purpose
Visual Studio	Writing, building and debugging the C# code
.NET 8 SDK	Compiler and runtime
Inno Setup 6	Making the Windows installer
Git + GitHub	Version control, code hosting, releases and app updates
PowerShell / batch files	Build script, test scripts and helpers
Windows Security	Testing alongside Defender and Controlled Folder Access

13. Limits of Kavach

Kavach is a real working antivirus, but as a student project it has limits that big commercial products avoid:

- **No kernel driver**: Kavach scans files a few seconds after they are saved, not before they open. Blocking before opening needs a Microsoft-signed minifilter driver.
- **No code-signing certificate**: Windows shows “Unknown publisher” on install.
- **Not registered in Windows Security Center**: only Microsoft partners can do that, so keep Defender on.
- **Smaller signature database**: hundreds of rules instead of millions of signatures; no cloud reputation.
- **Only part of YARA**: rules that need YARA modules (pe, math …) are skipped.

14. Key terms

Term	Meaning
Hash (SHA-256)	A unique 64-character fingerprint of a file. Change one byte and the hash changes completely.
Signature	Something that identifies malware: a hash or a rule.
YARA rule	A pattern describing a malware family, written in the YARA language.
Heuristics	Spotting new malware by suspicious features instead of exact matches.
PE file	Portable Executable — the format of Windows .exe and .dll files.
Entropy	How random data looks (0–8). Packed or encrypted code is close to 8.
Authenticode	Microsoft's digital signature system for programs.
Quarantine	A safe, encrypted place where threats are kept so they cannot run.
False positive	A safe file wrongly flagged as malware.
Ransomware	Malware that encrypts your files and demands payment.
Fileless attack	An attack that runs only in memory (e.g. PowerShell) without saving a file.
Windows service	A program that runs in the background without a window, even before login.
Named pipe	A Windows channel for two programs on the same PC to talk.
P/Invoke	How C# calls functions in Windows DLLs.
EICAR	A harmless standard test file every antivirus detects on purpose.